InfoCord

Name:

InfoCord *(tentative)*

Authors:

Pranav Madan

Context:

Modern note-taking and planning applications often store and process sensitive user data in ways that allow service providers to access, analyze, or monetize that information. This creates privacy concerns, especially for users handling personal, academic, or strategic content.

InfoCord is designed as a **privacy-preserving alternative**, where user data is **never readable by the server**. The system emphasizes **end-to-end encryption (E2EE)**, minimal data collection, and user ownership of information.

Overview of Solution:

InfoCord is a web-based (and later mobile) note organization system where:

- Users create accounts and organize notes into categories
- All note content is **encrypted on the client before being sent to the server**
- The server stores only **encrypted data (ciphertext)**
- Decryption occurs only on the user's device

The system follows a strict architectural principle:

> The server acts only as a storage and synchronization layer. The client owns and processes all sensitive data.

Tech Stack (with justification):

Backend: Flask (Python)

- Lightweight and flexible
- Full control over authentication and API logic
- Avoids unnecessary abstraction for MVP

Database: PostgreSQL (Neon for deployment)

- Reliable relational database
- Strong support for structured data (users, notes, categories)
- Neon allows scalable, serverless PostgreSQL

Frontend: HTML / CSS / JavaScript

- Minimal overhead for MVP
- Direct integration with Web Crypto API

Encryption: Web Crypto API

- Built into browsers
- Supports AES-GCM and PBKDF2
- Secure and performant without external dependencies

Password Hashing: Werkzeug / bcrypt

- Secure password storage
- Industry standard approach

Local Storage (Web): IndexedDB (Post-MVP)

- Enables offline functionality
- Stores encrypted data and sync queue

Advantages:

- **End-to-End Encryption (E2EE)**: Server cannot read user data
- **Zero-Knowledge Design**: No content inspection or tracking
- **User Data Ownership**: Encryption keys never leave client
- **Simple, Controlled Architecture**: Avoids overengineering

- **Scalable Foundation**: Ready for mobile and offline expansion

---

MVP Development Plan:

Phase 0 — Environment Setup

- Install Python (3.10+)
- Install PostgreSQL
- Initialize Git repository
- Create virtual environment
- Install dependencies:
    - Flask
    - SQLAlchemy
    - psycopg2-binary
    - Werkzeug
    - Flask-Migrate

Phase 1 — Backend Foundation (Local Development)

- Set up Flask project structure
- Configure PostgreSQL connection
- Define models:
    - User
    - Category
    - Note
- Run migrations

Phase 2 — Authentication (Core MVP Requirement)

- Implement:
    - Signup endpoint
    - Login endpoint
- Add:
    - Password hashing (bcrypt/Werkzeug)
    - Email uniqueness enforcement
    - HTTP-only cookie sessions
- Security additions:
    - Rate limiting (Flask-Limiter)

    - Temporary account lockout

Phase 3 — Core CRUD Functionality (Plaintext First)

- Categories:
    - Create, read, update, archive
- Notes:
    - Create, read, update, archive
- Add validation:
    - Max 10,000 characters
    - Required fields
- Ensure full end-to-end functionality locally

Phase 4 — Client-Side Encryption Integration

- Implement PBKDF2 key derivation (Web Crypto API)
- Derive encryption key from user password
- Encrypt note content before sending to backend
- Decrypt notes after retrieval
- Store:
    - ciphertext
    - IV (initialization vector)
- Note: If you forget your password at this stage, it will not be recovered as this is an additional level of complexity.

Phase 5 — Version-Based Sync System

- Add `version` field to notes
- On update:
    - server validates version match
    - rejects conflicts
- Client handles conflict resolution

Phase 6 — Stability & Error Handling

- Improve API error responses
- Add logging (server-side)

- Handle edge cases:
    - failed requests
    - invalid states

Phase 7 — Deployment (Optional for MVP Validation)

- Backend → Render
- Database → Neon
- Ensure:
    - HTTPS enabled
    - Secure cookies working
    - Environment variables configured

Post-MVP Plan:

- Implement offline support:
    - IndexedDB caching
    - Sync queue system
- Develop mobile apps (Flutter):
    - Android first
    - iOS second
- Secure key storage:
    - iOS Keychain
    - Android Keystore
- Add recovery mechanisms:
    - recovery keys (since password = encryption key)
- Improve sync:
    - retry logic
    - better conflict handling
- Explore advanced features:
    - encrypted search
    - optional AI features (privacy-preserving)
- Enhance UX:
    - loading states
    - sync indicators

Important Disclaimer:

Because InfoCord uses end-to-end encryption:

- Passwords are used to derive encryption keys
- If a password is lost, **data cannot be recovered**
- The system cannot access or restore user content
